

3.3

Model Training with Stochastic Gradient Descent

Part 4: Gradient Descent Vs Stochastic Gradient Descent

Sebastian Raschka and the PyTorch Lightning Team

The Gradient Descent Algorithm

For each training epoch:

`overall_loss` $L = 0$

For each training example:

`output` = $\sigma(z)$

`overall_loss` $L = \text{overall_loss } L + l(\text{output}, \text{label})$

`overall_loss` $L = \text{overall_loss } L / n$

compute $\nabla L_{\mathbf{w}}$ and ∇L_b

$\mathbf{w} = \mathbf{w} + \alpha \nabla L_{\mathbf{w}}$

$b = b + \alpha \nabla L_b$

Update parameters once per epoch

The **Stochastic** Gradient Descent Algorithm

Stochastic Gradient Descent and Minibatch Learning

For each training epoch:

overall_loss $L = 0$

For each training example:

output = $\sigma(z)$

overall_loss $L = \text{overall_loss } L + l(\text{output}, \text{label})$

$L = l(\text{output}, \text{label})$

overall_loss $L = \text{overall_loss } L / n$

compute $\nabla L_{\mathbf{w}}$ and ∇L_b

$\mathbf{w} = \mathbf{w} + \alpha \nabla L_{\mathbf{w}}$

$b = b + \alpha \nabla L_b$

The Stochastic Gradient Descent Algorithm

For each training epoch:

overall_loss $L = 0$

For each training example:

output = $\sigma(z)$

overall_loss $L = \text{overall_loss} + l(\text{output}, \text{label})$

$L = l(\text{output}, \text{label})$

overall_loss $L = \text{overall_loss} / n$

compute $\nabla L_{\mathbf{w}}$ and ∇L_b

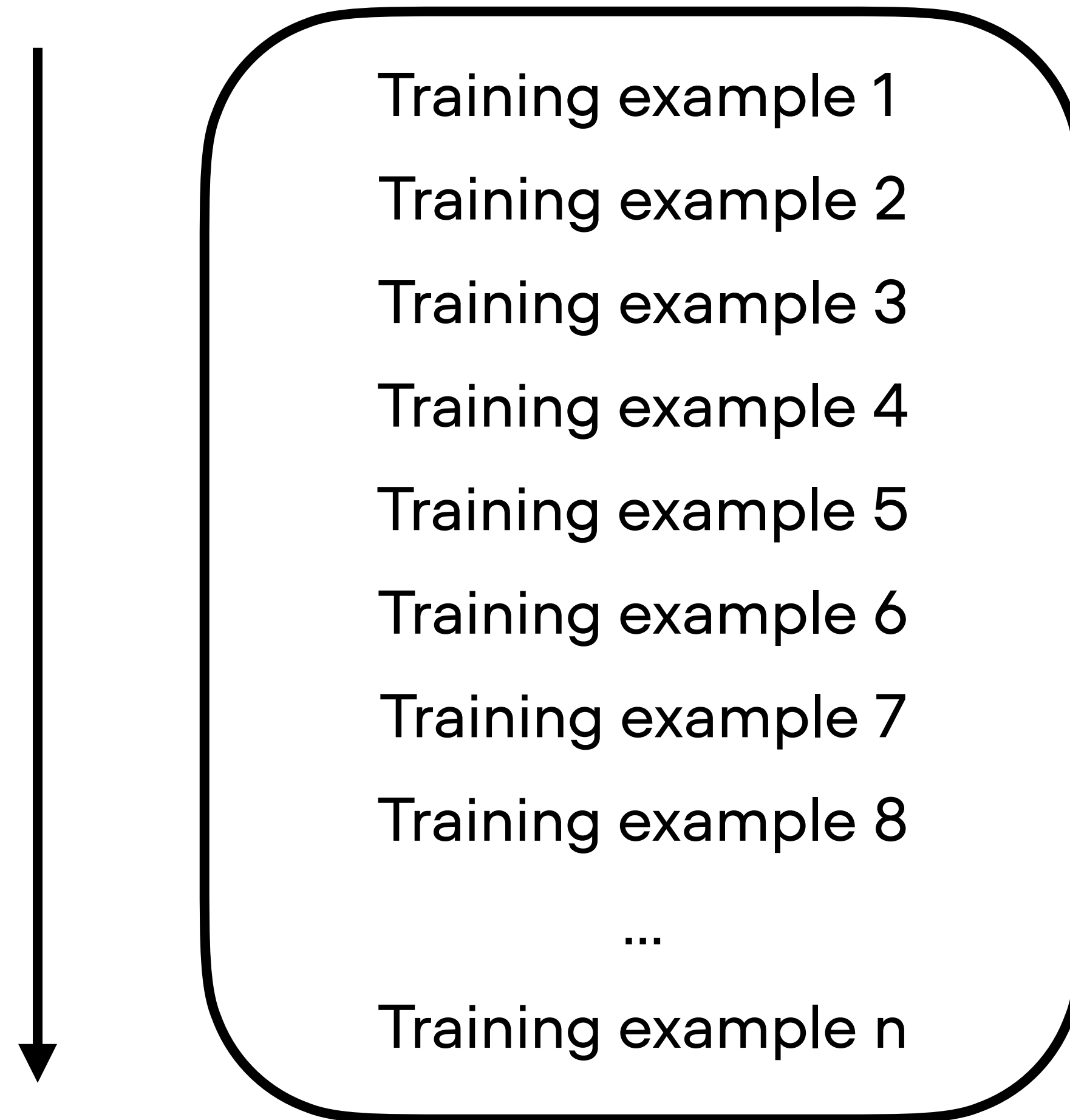
$\mathbf{w} = \mathbf{w} + \alpha \nabla L_{\mathbf{w}}$

$b = b + \alpha \nabla L_b$

Computes the loss and updates
for each training example

The Gradient **Stochastic** Descent Algorithm with Minibatches

1 update per epoch
(gradient descent)



n updates per epoch
(**stochastic gradient
descent**)



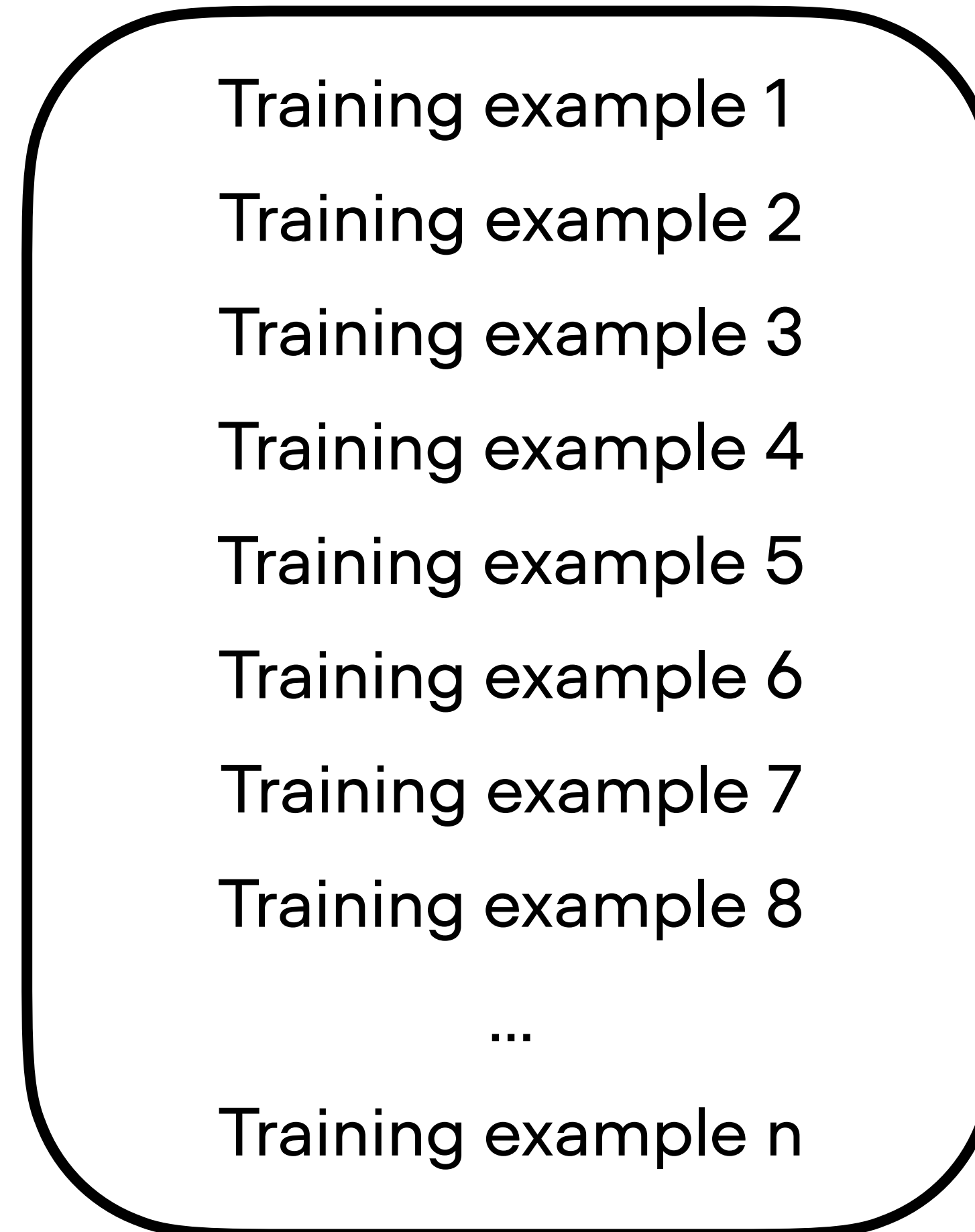
Training example 1
Training example 2
Training example 3
Training example 4
Training example 5
Training example 6
Training example 7
Training example 8
...
Training example n

m updates per epoch
(minibatch gradient
descent)



Training example 1
Training example 2
Training example 3
Training example 4
Training example 5
Training example 6
Training example 7
Training example 8
...
Training example n

m updates per epoch
(minibatch gradient descent)



Typical minibatch sizes are

$$2^4 = 16$$

$$2^5 = 32$$

...

$$2^8 = 512$$

Minibatch Gradient Descent Algorithm

For each training epoch:

For each minibatch:

overall_loss $L = 0$

For each training example in minibatch:

output = $\sigma(z)$

overall_loss $L = \text{overall_loss } L + l(\text{output}, \text{label})$

overall_loss $L = \text{overall_loss } L / n$

compute $\nabla L_{\mathbf{w}}$ and ∇L_b

$\mathbf{w} = \mathbf{w} + \alpha \nabla L_{\mathbf{w}}$

$b = b + \alpha \nabla L_b$

Minibatch Gradient Descent Algorithm

For each training epoch:

For each minibatch:

overall_loss $L = 0$

For-loop can be
replaced with matrix
multiplication

For each training example in minibatch:

output = $\sigma(z)$

overall_loss $L = \text{overall_loss } L + l(\text{output}, \text{label})$

overall_loss $L = \text{overall_loss } L / n$

compute $\nabla L_{\mathbf{w}}$ and ∇L_b

$\mathbf{w} = \mathbf{w} + \alpha \nabla L_{\mathbf{w}}$

$b = b + \alpha \nabla L_b$

The Gradient **Stochastic** Descent Algorithm with Minibatches

Less noisy loss compared to stochastic gradient descent with
1 training example

The Gradient **Stochastic** Descent Algorithm with Minibatches

Faster than gradient descent because more than 1 update per epoch

The Gradient **Stochastic** Descent Algorithm with Minibatches

Better GPU utilization than stochastic gradient descent by using
linear algebra concepts (matrix multiplication)

Computing Gradients Automatically with PyTorch